

8 단계. 기술 스택(개발 스펙) 추천

각 단계마다 [프롬프트 예시] → [모의 예시] → [학생용 워크시트] 순서로 구성·개인별 작성

이름	연희
----	----

① PRD 기반 스택 추천받기

기술 스택은 프론트엔드·백엔드·데이터베이스·배포 방식을 총총이 쌓아올린 조합입니다. PRD 를 참고해 이 프로젝트에 맞는 스택 후보를 추천받습니다.

프롬프트 예시

7 단계에서 작성한 PRD 파일을 첨부해줘.
이 내용을 참고해서, 이 서비스에 적합한 개발 스펙(기술 스택)을 추천해줘.
프론트엔드 / 백엔드 / 데이터베이스 / 배포 방식을 각각 후보 2~3 개씩 제시하고,
왜 이 프로젝트(1 인 개발자, 바이브 코딩, 비개발자 학습자)에 적합한지 이유를 설명해줘.
코드는 작성하지 말고 구조와 이유 위주로 설명해줘.

모의 예시 (냉장고 파수꾼)

프론트엔드: React(Vercel 배포) — 무료 배포가 쉽고 AI 코드 생성 도구와 호환성이 높음

백엔드/DB: Supabase — DB·인증·API 가 통합되어 있어 백엔드 코드를 따로 작성하지 않아도 됨

배포: Vercel — React 프로젝트와 궁합이 좋고 별도 서버 관리 없이 바로 배포 가능

⇒ 학생용 워크시트 — 내 프로젝트로 채우기

층(Layer)	후보 1	후보 2	선택 이유
프론트엔드	React (Vite)	Next.js	이미 참고한 Weather_cloth 앱이 React+TS+Vite 로 만들어져 있어 화면(분기·폼·리스트) 구조를 그대로 참고 가능. SSR/SEO 가 필요 없는 개인용 웹앱이라 Vite 로 충분
백엔드/DB	Supabase	자체 Node.js 서버 + PostgreSQL	family·Weather_cloth·my_refrigerator 세 참고 앱이 모두 이미 Supabase 를 쓰고 있었고, my_refrigerator PRD 에는 '여러 앱이 공유하는 Supabase 프로젝트'라는 전제까지 있어 새 프로젝트도 같은 방식이 익숙함

층(Layer)	후보 1	후보 2	선택 이유
배포	Vercel	Netlify	React/Vite 프로젝트와 궁합이 좋고, my_refrigerator도 이미 Vercel(vercel.json)로 배포 중이라 배포 경험이 있음

② 경량/확장 버전 비교표로 다듬기

프롬프트 예시

추천받은 후보들을 경량 버전(빠른 MVP 완성 우선)과 확장 버전(향후 기능 확장 고려)으로 나눠서 비교표로 정리해줘.
각 버전이 어떤 타겟 사용자·상황(페르소나, 발표 일정 등)에 적합한지도 함께 설명해줘.

⇒ 학생용 워크시트 — 내 프로젝트로 채우기

층(Layer)	경량 버전	확장 버전	우리 프로젝트에 맞는 선택
프론트엔드	React + Vite, 정적 SPA (SSR 없음)	Next.js, SSR/SEO 대응	경량 — 로그인 없는 가족 내부용 웹앱이라 SEO·SSR 이 필요 없음
백엔드/DB	Supabase 기본 테이블 + RLS 만 사용	Supabase + Edge Functions (추후 AI 레시피 재도입 시)	경량 — 레시피 기능을 냉장고 연동 없이 이미 최소화했으므로(4 단계 결정) 별도 서버 로직 불필요
배포	Vercel 무료 티어	Vercel Pro (트래픽 증가 대비)	경량 — 가족 단위 소규모 사용이라 무료 티어로 충분

최종 선택 이유 (페르소나·프로젝트 상황과 연결)

경량 버전(React+Vite / Supabase 기본기능 / Vercel 무료)을 선택함.
1 인 개발·바이브코딩 학습 목적이라, 확장 버전의 SSR·Edge Function 은 지금 검증 단계에서는 과한 투자임(Day1 리스크: '기능 영역이 늘면서 완성도 분산').
4 단계에서 냉장고 재료인식 AI 기능을 '냉장고 연동 없는 레시피 2 개'로 이미 축소 결정했기 때문에, 별도 AI API 서버 (my_refrigerator 의 Flask+OpenRouter 방식)가 필요 없어짐.
페르소나 기술 친숙도('설정이 복잡하거나 매번 손으로 입력해야 하는 건 금방 안 씀')를 고려하면, 개발 쪽도 최대한 단순한 구조를 유지하는 게 유지보수 측면에서 안전함.

③ 검증 — PRD·페르소나와 대조하기

기술 스택은 유행이나 익숙함으로 고르기 쉬운 항목입니다. 마지막으로 이 선택이 PRD 의 비기능 요구사항, 그리고 페르소나의 기술 친숙도와 실제로 맞는지 확인합니다.

프롬프트 예시

방금 추천된 기술 스택이 7 단계 PRD 의 비기능 요구사항, 그리고 3 단계 페르소나의 기술 친숙도와 실제로 맞는지 다시 확인해줘. 맞지 않는 부분이 있다면 어떤 대안이 더 적합한지 제안해줘.

⇒ 학생용 워크시트 — 내 프로젝트로 채우기

확인 항목 (비기능 요구사항 / 페르소나 기술 친숙도)	추천 스택과 일치 여부 (O/X)	조정 필요 사항
PRD 비기능요구사항: 온보딩 단순성 유지('옷장 등록 없이 매일 자동으로 알려주는 점')	O	React+Supabase 조합으로 별도 회원가입 절차 없이 구성원 선택만으로 진입 가능해 부합
PRD 비기능요구사항: 부부 완료 현황 실시간성(Day5 에서 '검증 안 됨'으로 남김)	△	Supabase Realtime 으로 기술적으로는 구현 가능하나, 정말 필요한지는 PRD 에서도 미확정 상태 — 우선 새로고침 방식으로 시작하고 필요시 Realtime 추가하는 조정 필요
페르소나 기술 친숙도: '설정이 복잡하거나 매번 손으로 입력해야 하는 건 금방 안 씬'	△	웹앱은 앱스토어 설치 없이 링크로 바로 접속 가능해 설치 장벽은 낮으나, 네이티브 앱 대비 '홈 화면 아이콘' 등 재접속 편의성은 별도 설정(PWA) 필요
PRD 비교: 기술스택은 이전까지 '대기' 상태로 자료없음	O	이번 8 단계에서 확정 — PRD ⑦ 비기능요구사항의 '기술스택은 대기 상태' 문구도 함께 업데이트 필요